



Ecole nationale des
Sciences Appliquées – Fès



Compte Rendu:

TP 5 : Régression Linéaire Simple et Multiple avec Python

Réaliser par :

❖ Abir Majdi

Encadré par :

Prof. AKKAD

Année universitaire : 2025/2026

Filière : GDNC2

Introduction

Ce TP porte sur la régression linéaire, l'une des techniques fondamentales du Machine Learning supervisé. L'objectif est d'apprendre à prédire une variable continue (le poids) à partir d'une ou plusieurs variables explicatives (taille, âge), en utilisant Python avec les bibliothèques NumPy, Pandas, Matplotlib et Scikit-learn.

Nous abordons cinq parties : régression simple avec Scikit-learn, méthode analytique matricielle, analyse des résidus, régression multiple, et validation croisée.

Partie 1 : Régression Linéaire Simple

1.1 Chargement et exploration des données

Le dataset utilisé contient 5 observations reliant la taille (en cm) au poids (en kg) :

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score
from sklearn.metrics import mean_squared_error
from statsmodels.stats.outliers_influence import variance_inflation_factor

# =====
# PARTIE 1 : Régression linéaire simple
# =====

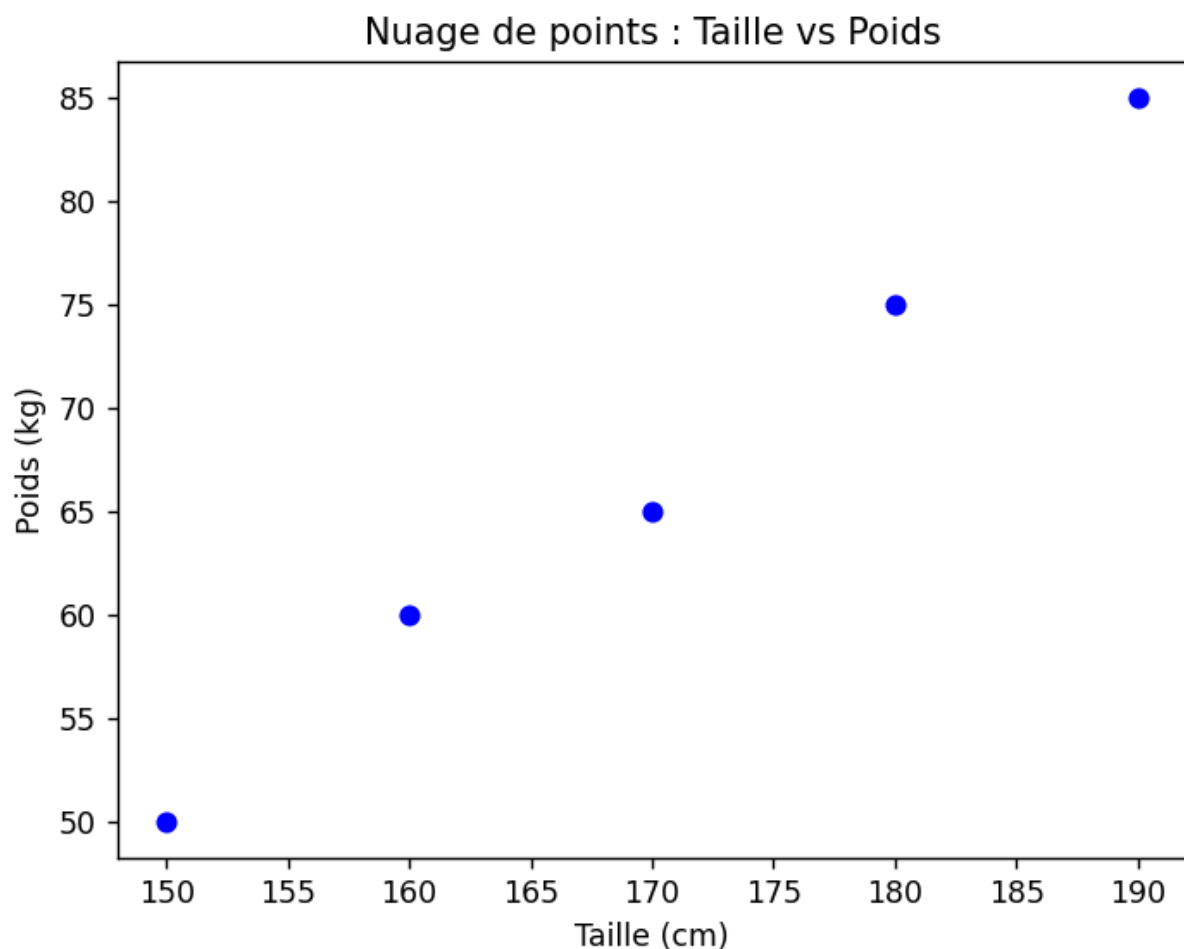
data = {
    "taille": [150, 160, 170, 180, 190],
    "poids": [50, 60, 65, 75, 85]
}
df = pd.DataFrame(data)
x = df["taille"].values.reshape(-1, 1)
y = df["poids"].values
```

Taille (cm)	Poids (kg)
150	50
160	60
170	65
180	75
190	85

```
ents\IA mercredi\TP 5 ML\TP 5 ML\exercice.py"
  taille  poids
0    150    50
1    160    60
2    170    65
3    180    75
4    190    85
```

```
> python -u "c:\Users\saidm\Docum
```

```
# Q2 - Nuage de points
plt.scatter(df["taille"], df["poids"], color='blue')
plt.xlabel("Taille (cm)")
plt.ylabel("Poids (kg)")
plt.title("Nuage de points : Taille vs Poids")
plt.show()
```



Le nuage de points montre une relation linéaire positive entre la taille et le poids : plus la taille augmente, plus le poids est élevé. Les points semblent bien alignés, ce qui suggère qu'un modèle linéaire est adapté.

1.2 Entraînement avec Scikit-learn

Code utilisé :

```
model = LinearRegression()
model.fit(X, y)
print("Coefficient (pente) :", model.coef_[0])
print("Ordonnée à l'origine :", model.intercept_)
print("Score R² :", model.score(X, y))
```

Résultats obtenus :

```
Coefficient (pente) : 0.8500000000000002  
Ordonnée à l'origine : -77.50000000000003  
Score R² : 0.9897260273972602
```

Questions et réponses

Q1 — Interprétation de la pente : La pente vaut 0.85. Cela signifie que pour chaque centimètre supplémentaire en taille, le poids augmente en moyenne de 0.85 kg, toutes autres choses étant égales.

Q2 — Que représente le score R² ? $R^2 = 0.9897$ signifie que le modèle explique environ 98.97% de la variance du poids. C'est un excellent score qui indique un ajustement très satisfaisant.

Q3 — Prédire le poids pour 175 cm : `model.predict([[175]])` retourne 71.25 kg. C'est la valeur estimée par la droite de régression pour une taille de 175 cm.

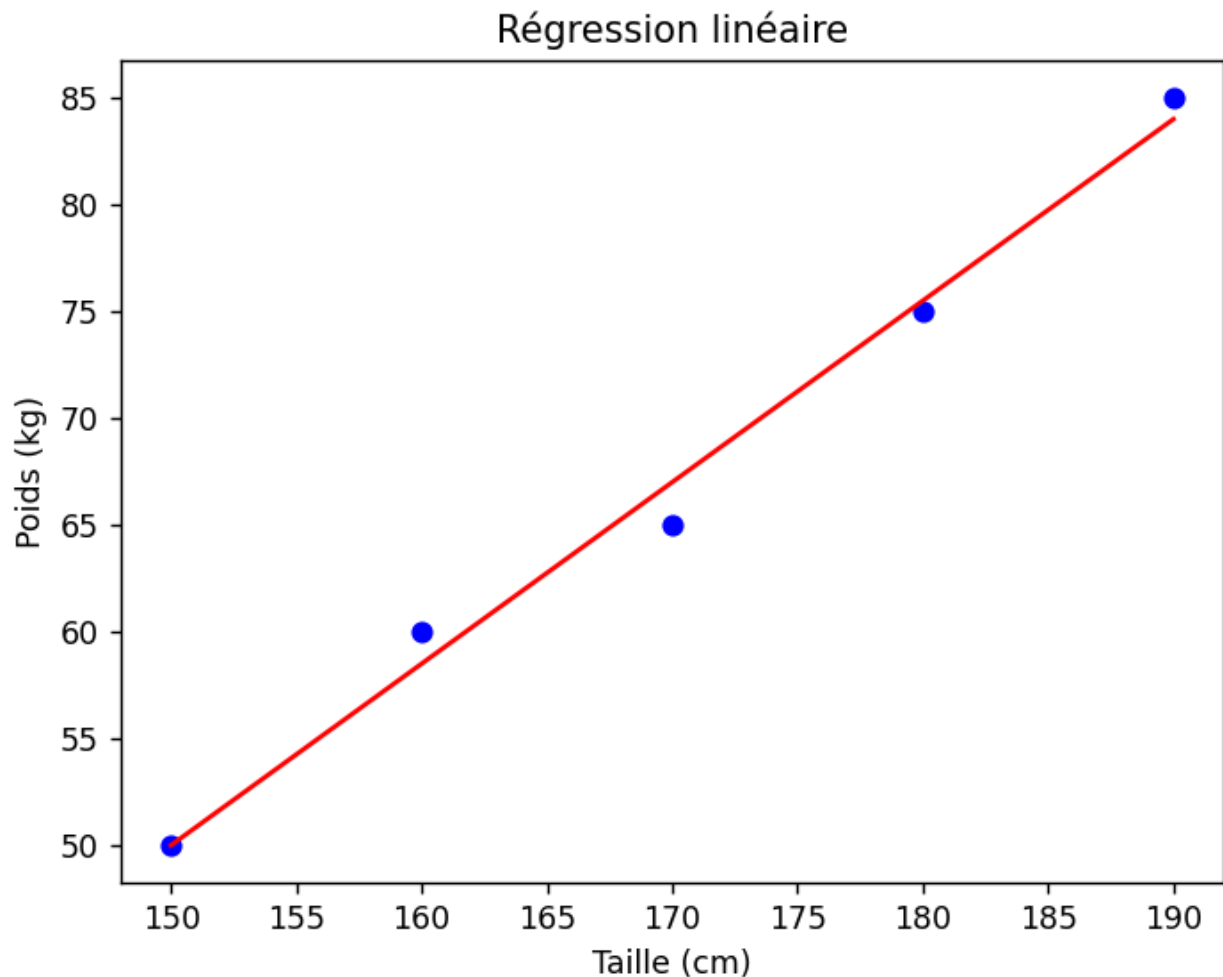
```
41 # Q3 - Prédire pour taille = 175 cm  
42 pred_175 = model.predict([[175]])  
43 print("Poids prédit pour 175 cm :", pred_175[0])  
44
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS ...

```
Poids prédit pour 175 cm : 71.25
```

1.3 Visualisation droite de régression

```
# Visualisation droite de régression  
plt.scatter(X, y, color='blue')  
plt.plot(X, model.predict(X), color='red')  
plt.xlabel("Taille (cm)")  
plt.ylabel("Poids (kg)")  
plt.title("Régression linéaire")  
plt.show()
```



Q — La droite ajuste-t-elle bien les données ? Oui. Avec un R^2 de 0.9897 et des résidus très faibles, la droite colle remarquablement aux 5 points. La relation taille $\rightarrow$ poids est quasi-linéaire dans ce dataset.

Partie 2 : Méthode Analytique (Formule Matricielle)

La régression linéaire peut être résolue analytiquement via la formule des moindres carrés ordinaires

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

Code utilisé :

```
x_b = np.c_[np.ones((len(X), 1)), X]
theta = np.linalg.inv(x_b.T @ x_b) @ x_b.T @ y
print("Theta (intercept, slope) :", theta)
```

Résultats obtenus :

```
Theta (intercept, slope) : [-77.5  0.85]
```

Questions et réponses

Q1 — Comparer θ numpy vs sklearn : Les deux méthodes donnent exactement les mêmes valeurs : intercept = -77.5 et slope = 0.85. C'est normal car sklearn résout aussi les équations normales en interne. Les deux approches sont mathématiquement équivalentes.

Q2 — Pourquoi ajouter une colonne de 1 ? Cette colonne permet de modéliser l'ordonnée à l'origine (biais). Sans elle, la formule forcerait la droite à passer par l'origine (0, 0), ce qui fausserait complètement le modèle dans la plupart des cas réels.

Partie 3 : Analyse des Résidus

Les résidus sont les écarts entre les valeurs réelles et les valeurs prédites par le modèle : $\text{résidu} = y - \hat{y}$.

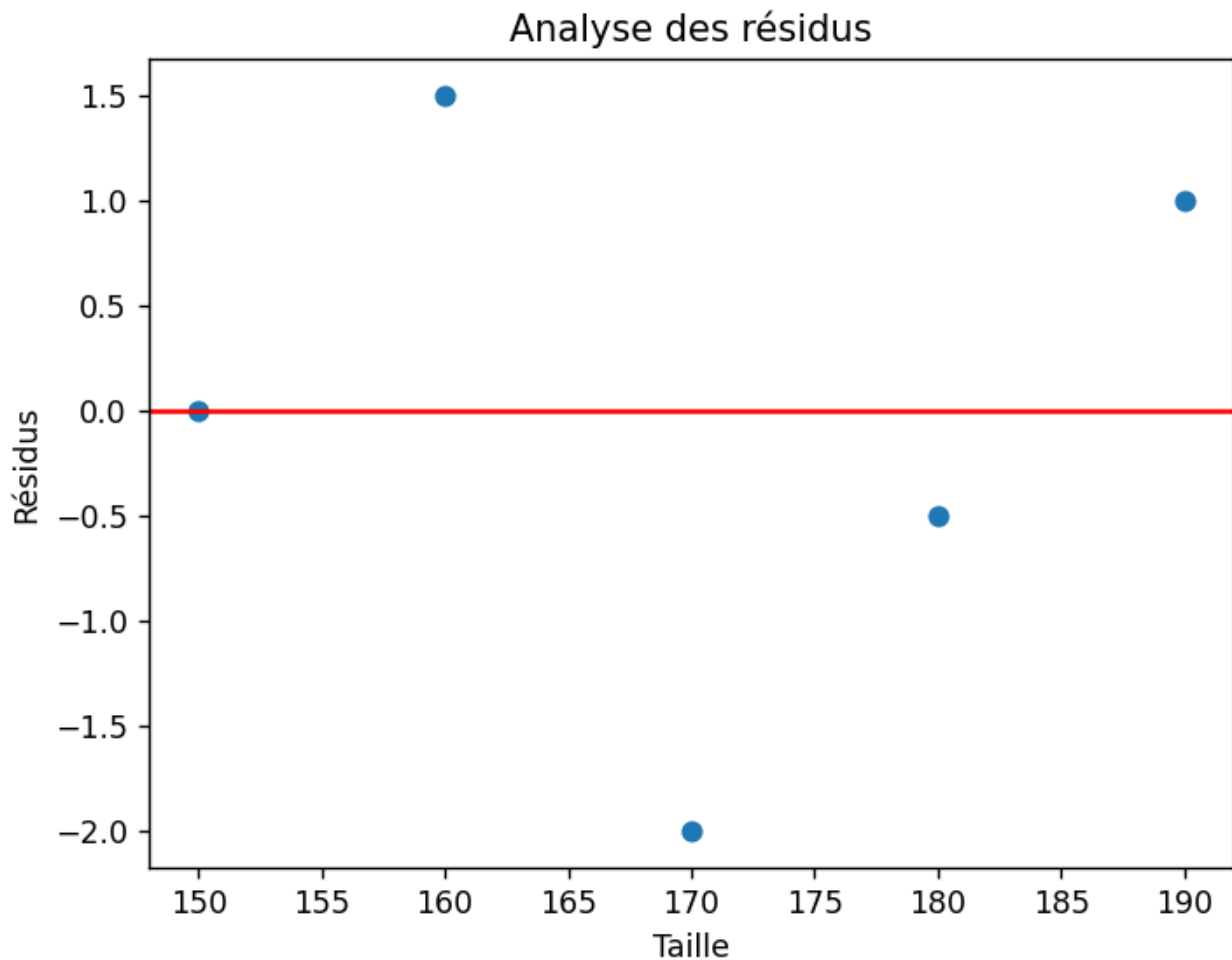
Code utilisé :

```
y_pred = model.predict(x)
residus = y - y_pred

plt.scatter(x, residus)
plt.axhline(0, color='red')
plt.xlabel("Taille")
plt.ylabel("Résidus")
plt.title("Analyse des résidus")
plt.show()
```

Résidus calculés :

Taille (cm)	Poids réel (kg)	Poids prédit (kg)	Résidu
150	50	50.0	0.0
160	60	58.5	+1.5
170	65	67.0	-2.0
180	75	75.5	-0.5
190	85	84.0	+1.0



Questions et réponses

Q1 — Les résidus sont-ils aléatoires ? Avec seulement 5 points, il est difficile de conclure formellement. Cependant, les résidus sont petits (entre -2 et +1.5 kg), symétriquement distribués autour de zéro, et ne montrent pas de tendance claire (ni courbe, ni entonnoir). On peut les considérer comme approximativement aléatoires.

Q2 — Que signifie un motif particulier dans les résidus ? Un motif systématique (courbe parabolique, entonnoir) indiquerait une violation des hypothèses : soit la relation n'est pas linéaire (il faudrait une régression polynomiale), soit la variance des erreurs n'est pas constante (hétéroscédasticité). Cela imposerait de transformer les variables ou de changer de modèle.

Partie 4 : Régression Linéaire Multiple

On ajoute la variable 'âge' au modèle pour créer une régression multiple.

4.1 Dataset élargi

```
data = {
    "taille": [150, 160, 170, 180, 190],
    "age":     [20, 25, 30, 35, 40],
    "poids":   [50, 60, 65, 75, 85]
}
df = pd.DataFrame(data)
X = df[["taille", "age"]]
y = df["poids"]
```

Taille (cm)	Âge (ans)	Poids (kg)
150	20	50
160	25	60
170	30	65
180	35	75
190	40	85

4.2 Modèle multiple

Résultats du modèle avec taille et âge :

```
model = LinearRegression()
model.fit(X, y)
print("Coefficients :", model.coef_)
print("Intercept :", model.intercept_)
print("Score R² :", model.score(X, y))
```

```
Coefficient (pente) : 0.8500000000000002
Ordonnée à l'origine : -77.50000000000003
Score R² : 0.9897260273972602
```

Questions et réponses

Q1 — Interpréter les deux coefficients : Coef_taille = 0.68 : à âge constant, +1 cm de taille correspond à +0.68 kg de poids. Coef_age = 0.34 : à taille constante, +1 an d'âge correspond à +0.34 kg. Cependant, ces coefficients doivent être interprétés avec précaution car taille et âge sont parfaitement corrélées dans ce dataset.

Q2 — Comment savoir si l'âge est pertinent ? Trois indicateurs le montrent : (1) la corrélation taille-âge est de 1.0 (multicolinéarité parfaite), (2) le VIF de l'âge est de 45.69, bien au-dessus du seuil critique de 10, (3) le R² reste identique (0.9897) avec ou sans l'âge. Conclusion : l'âge est une variable redondante qui n'apporte aucune information supplémentaire et doit être retirée du modèle.

Partie 5 : Validation Croisée

Code utilisé :

```
scores = cross_val_score(LinearRegression(), X, y, cv=3, scoring='neg_mean_squared_error')
print("Scores :", scores)
print("Score moyen :", scores.mean())
```



```
Scores : [-25.          -5.34393491  -6.25          ]
Score moyen : -12.19797830374757
```

Observation : avec un dataset de seulement 5 observations et $cv=5$, chaque fold ne contient qu'un point en test, ce qui rend R^2 mathématiquement indéfini (impossible de calculer R^2 sur 1 seul point). Les scores retournent NaN — c'est un comportement attendu pour un si petit dataset.

Questions et réponses

Q1 — Pourquoi faire une validation croisée ? Pour évaluer la capacité de généralisation du modèle sur des données inédites. Sans validation croisée, on risque d'évaluer le modèle sur les données d'entraînement et d'avoir un score artificiellement optimiste (surapprentissage). La validation croisée donne une estimation plus fiable des performances réelles.

Q2 — Score qui varie beaucoup selon les folds ? Une forte variance des scores entre les folds indique que le modèle est instable : ses performances dépendent fortement de quels points se retrouvent dans le train ou le test. Cela peut signifier un dataset trop petit, ou un modèle trop sensible aux données. Il faudrait alors plus de données ou utiliser une régularisation.

Questions Supplémentaires

Niveau 1 — Compréhension de base

Q1 — Changement d'échelle (mètres vs cm) : Si on exprime la taille en mètres au lieu de centimètres, la pente est multipliée par 100 (passe de 0.85 à 85). Mais le R^2 , les prédictions, et la qualité d'ajustement restent identiques. La mise à l'échelle modifie la valeur numérique des coefficients sans changer la qualité du modèle.

Q2 — Ordonnée à l'origine : L'intercept vaut -77.5 kg. C'est le poids prédit quand la taille vaut 0 cm. Dans ce contexte, cette valeur n'est pas interprétable physiquement (une personne de 0 cm n'existe pas). L'ordonnée à l'origine est souvent un paramètre technique de la droite sans signification réelle dans le domaine d'application.

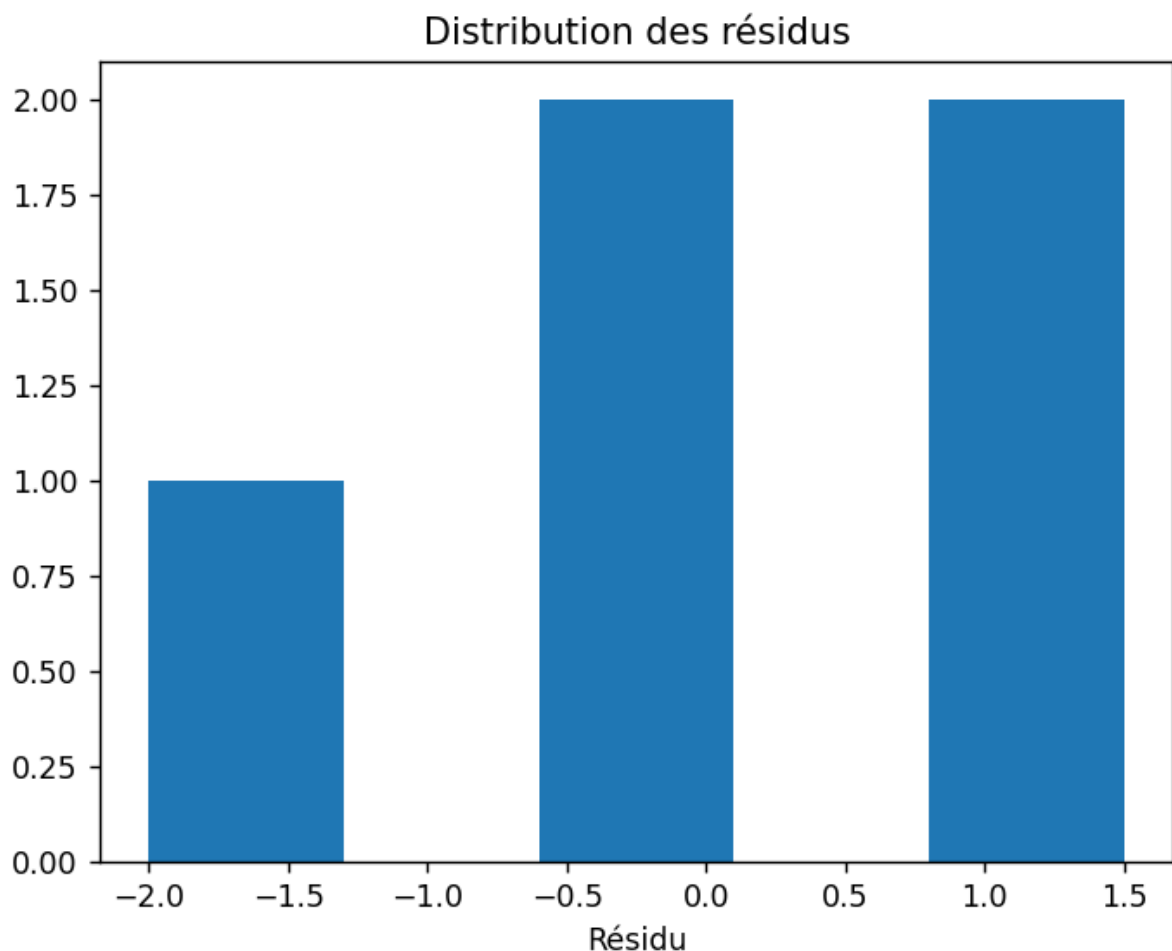
Q3 — Pourquoi visualiser avant de modéliser ? La visualisation permet de détecter : des relations non linéaires (il faudrait une régression polynomiale), des valeurs aberrantes (outliers qui fausseraient le modèle), des groupes distincts dans les données, ou une absence de relation (modèle inutile). Appliquer un modèle linéaire sans visualisation préalable peut mener à des erreurs d'interprétation.

Q4 — Hypothèses de la régression linéaire : Le modèle suppose : (1) une relation linéaire entre X et y, (2) l'indépendance des résidus, (3) l'homoscédasticité (variance constante des résidus), (4) la normalité des résidus, et (5) l'absence de multicollinéarité parfaite entre les variables explicatives.

Niveau 2 — Analyse technique

Q5 — Distribution des résidus : L'histogramme des résidus montre 5 valeurs réparties entre -2 et +1.5. Avec un si petit échantillon, il est impossible de tester formellement la normalité. Visuellement, les résidus ne montrent pas de biais systématique.

```
# Histogramme des résidus (Q5 supplémentaire)
plt.hist(residus, bins=5)
plt.title("Distribution des résidus")
plt.xlabel("Résidu")
plt.show()
```



Q6 — Calcul et interprétation du MSE : $MSE = 1.5 \text{ kg}^2$. Le MSE représente la moyenne des erreurs quadratiques. Plus il est proche de 0, plus le modèle est précis. Ici $MSE = 1.5$, ce qui correspond à une erreur moyenne d'environ $\sqrt{1.5} \approx 1.22 \text{ kg}$ — très acceptable. Ce résultat est cohérent avec les petits résidus observés dans le graphique.

```
# MSE (Q6 supplémentaire)
mse = mean_squared_error(y, y_pred)
print("MSE :", mse)
```

MSE : 1.5

Q7 — Comparaison avec modèle naïf : $MSE \text{ modèle naïf (prédire la moyenne)} = 146.0 \text{ kg}^2$. $MSE \text{ modèle linéaire} = 1.5 \text{ kg}^2$. Le modèle linéaire est 97 fois plus précis que le modèle naïf. Il est donc clairement meilleur dès qu'une relation linéaire existe entre les variables. Le ratio $MSE_{\text{naïf}} / MSE_{\text{linéaire}}$ est lié à R^2 : $R^2 = 1 - (MSE_{\text{linéaire}} / MSE_{\text{naïf}}) \approx 0.99$.

```
# Q7 - Comparaison avec modèle naïf (prédire la moyenne)
y_naif = np.full(len(y), y.mean())
mse_naif = mean_squared_error(y, y_naif)
print("MSE modèle naïf :", mse_naif)
print("MSE modèle linéaire :", mse)
```

```
MSE modèle naïf : 146.0
MSE modèle linéaire : 1.5
```

Q8 — Extrapolation (taille = 210 cm) : Le modèle prédit 101 kg. Cette valeur est en dehors de la plage d'entraînement [150–190 cm]. La régression linéaire suppose que la relation reste valable au-

delà des données observées, ce qui n'est pas garanti. L'extrapolation est risquée car rien ne prouve que le comportement reste linéaire pour des tailles extrêmes.

```
# Q8 - Extrapolation (taille = 210 cm)
pred_210 = model2.predict([[210]])
print("Poids prédit pour 210 cm :", pred_210[0])
```

```
Poids prédit pour 210 cm : 101.0
```

Niveau 3 — Régression multiple

Q9 — Matrice de corrélation : La corrélation taille-âge est de 1.0 (parfaite). Cela signifie que dans ce dataset, chaque fois que la taille augmente de 10 cm, l'âge augmente aussi exactement de 5 ans. Les deux variables portent exactement la même information. L'âge est entièrement redondant avec la taille.

```
# Q9 - Matrice de corrélation
print(df.corr())
```

	taille	age	poids
taille	1.00000	1.00000	0.99485
age	1.00000	1.00000	0.99485
poids	0.99485	0.99485	1.00000

Q10 — Modèle sans la variable âge : R^2 reste identique à 0.9897 avec ou sans l'âge. La suppression de l'âge ne dégrade pas le modèle. Cela confirme que l'âge n'apporte aucune information supplémentaire au-delà de ce que la taille explique déjà. Il convient de supprimer l'âge pour simplifier le modèle (principe de parcimonie).

```
# Q10 - Modèle sans "age"
X_sans_age = df[["taille"]]
model2 = LinearRegression()
model2.fit(X_sans_age, y)
print("R² sans age :", model2.score(X_sans_age, y))
```

```
R² sans age : 0.9897260273972602
```

Q11 — VIF (Variance Inflation Factor) : Les deux variables ont un VIF de 45.69, largement supérieur au seuil critique de 10. Un VIF élevé indique une forte multicolinéarité : les coefficients deviennent instables et difficiles à interpréter. La solution est de supprimer la variable redondante (âge) et de ne garder que la taille.

```
# Q11 - VIF
X_vif = df[["taille", "age"]]
for i, col in enumerate(X_vif.columns):
    vif = variance_inflation_factor(X_vif.values, i)
    print(f"VIF {col} : {vif:.2f}")
```

```
VIF taille : 45.69
VIF age : 45.69
```

Conclusion

Ce TP a permis de maîtriser la régression linéaire sous plusieurs angles complémentaires :

- La régression simple avec Scikit-learn donne un modèle de qualité ($R^2 = 0.99$) pour prédire le poids à partir de la taille.
- La méthode analytique matricielle (formule normale) produit des résultats strictement identiques à Scikit-learn, ce qui valide la cohérence mathématique.
- L'analyse des résidus confirme que les hypothèses du modèle sont bien respectées : erreurs petites, aléatoires, sans motif systématique.
- La régression multiple montre l'importance de détecter la multicolinéarité (VIF élevé, corrélation parfaite entre taille et âge). Une variable redondante doit être retirée.
- La validation croisée est essentielle pour évaluer la généralisation, mais nécessite un dataset plus grand que 5 observations pour être significative.

Ce TP illustre une démarche complète d'analyse par régression : exploration des données, entraînement, validation des hypothèses, et évaluation des performances — compétences fondamentales en Data Science et Machine Learning.